Пројектовање синтаксног анализатора 2

Прављење синтаксног анализатора

- Можемо га правити ручно
  - Разни могући алгоритми. Разликују се по граматикама са којима могу да се носе и по ефикасности (меморијском и процесорском заузећу)
  - Типична ручна техника: Синтаксни анализатор са рекурзивним спуштањем
- Можемо га генерисати помоћу алата
  - Алат који на основу граматике генерише синтаксни анализатор
  - Један од таквих алата је **Bison** (део GNU-а)
  - **Bison** генерише анализатор који се заснива на LALR алгоритму

Граматика независна од контекста (контекстно слободна)

- Спецификација језика у облику низа продукција
  - Пример: језик праволинијских програма
    - Нетерминални симболи: *S*, *E* и *L*; почетни симбол је *S*
    - Терминални симболи: id print num , + ( ) := ;
    - Пример реченице у овом језику
      - id := num; id := id + (id := num + num, id)
      - Може потицати од следећег улаза (изворног кода):
      - a := 7; b := c + (d := 5 + 6, d)
      - Имена a,b,c и бројеви 5,6,7 су семантичке вредности симбола

$S \rightarrow S ; S$
$S \rightarrow \text{id} := E$
$S \rightarrow \text{print} (L)$

$E \rightarrow \text{id}$
$E \rightarrow \text{num}$
$E \rightarrow E + E$
$E \rightarrow (S, E)$

$L \rightarrow E$
$L \rightarrow L , E$

Изводи реченица и стабла анализе

- Постоји више извода исте реченица:

pera := 15; djole := 13 + pera

S → ***S*** **;** ***S***
S → **id :=** ***E***
S → **print (** ***L*** **)**

E → **id**
E → **num**
E → ***E*** **+** ***E***
E → **(*S*, *E*)**

L → ***E***
L → ***L*** **,** ***E***

Изводи реченица и стабла анализе

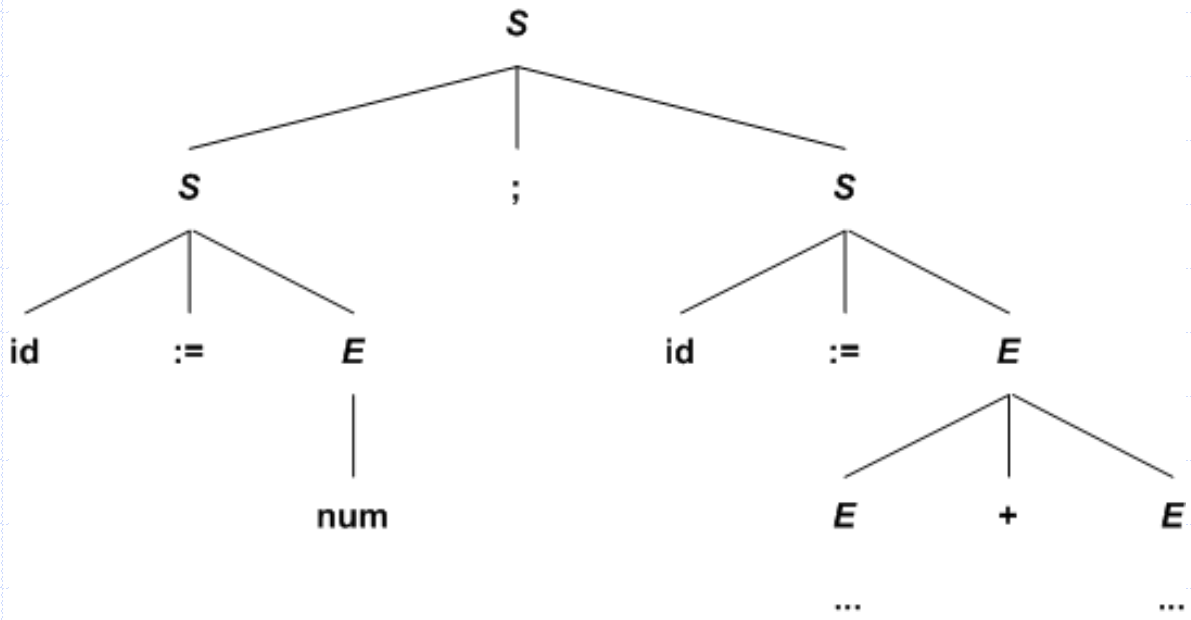
- Постоји више извода исте реченице, нпр.
  - Крајњи леви: увек се смењује крајњи леви нетерминал
  - Крајњи десни: увек се смењује крајњи десни нетерминал

S
S **;** ***S***
id := ***E*** **;** ***S***
id := num ; ***S***
id := num ; id := ***E***
id := num ; id := ***E*** **+** ***E***
...

Изводи реченица и стабла анализе

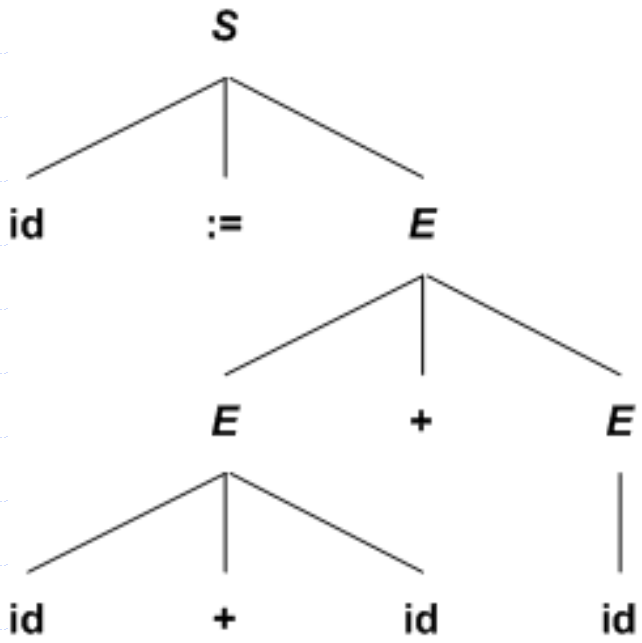
- Постоји више извода исте реченице, нпр.
  - Крајњи леви: увек се смењује крајњи леви нетерминал
  - Крајњи десни: увек се смењује крајњи десни нетерминал
- Стабло синтаксне анализе
  - Сваки симбол у изводу се спаја са симболом из ког је изведен
  - Два различита извода могу имати исто стабло синтаксне анализе

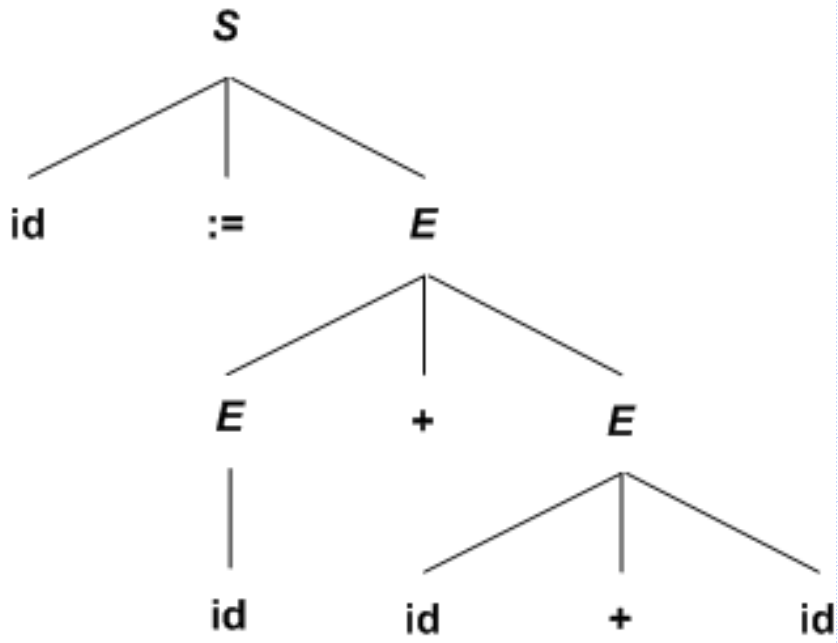
S
S **; *S***
id := *E* ; *S*
id := num ; *S*
id := num ; id := *E*
id := num ; id := *E* + *E*
...



Нејасне граматике

- Граматика је НЕЈАСНА ако може да произведе исту реченицу са два различита стабла
  - Нпр. граматика SLP-а је нејасна јер за реченицу „id:=id+id+id” постоје два стабла синтаксне анализе

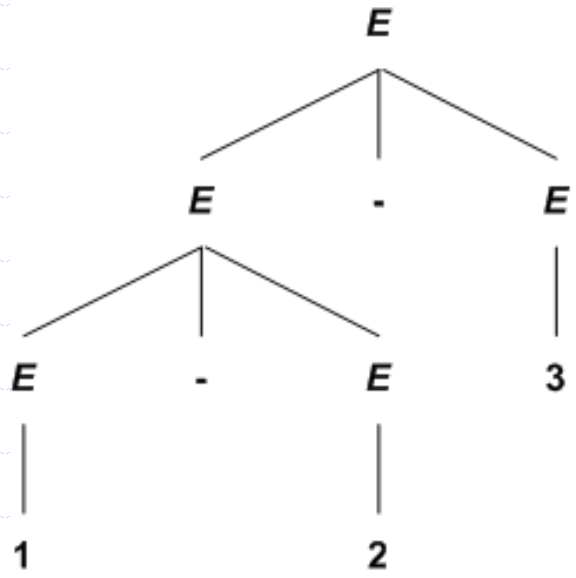


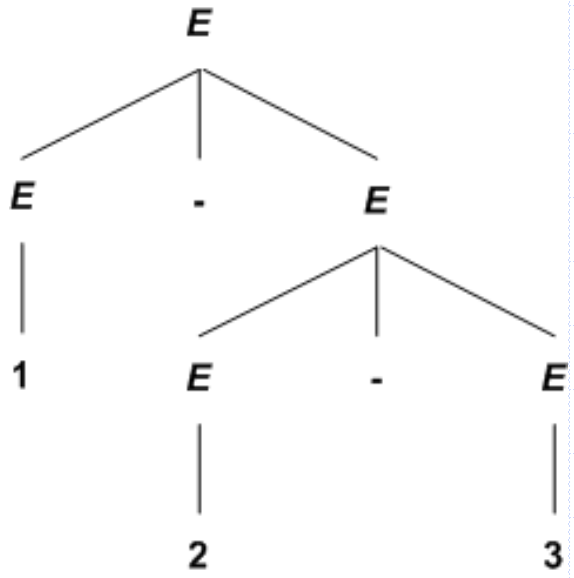


Проблем нејасне граматике: Различита значења!

- Пример 1: Граматика „Калкулатор 1”
  - Два стабла за реченицу „1-2-3”
- Ако се стабло користи за интерпретирање значења
  - Прво значење: (1-2)-3 = -4
  - Друго значење: 1-(2-3) = 2

$E \rightarrow \mathbf{id}$
$E \rightarrow \mathbf{num}$
$E \rightarrow E * E$
$E \rightarrow E / E$
$E \rightarrow E + E$
$E \rightarrow E - E$
$E \rightarrow (E)$

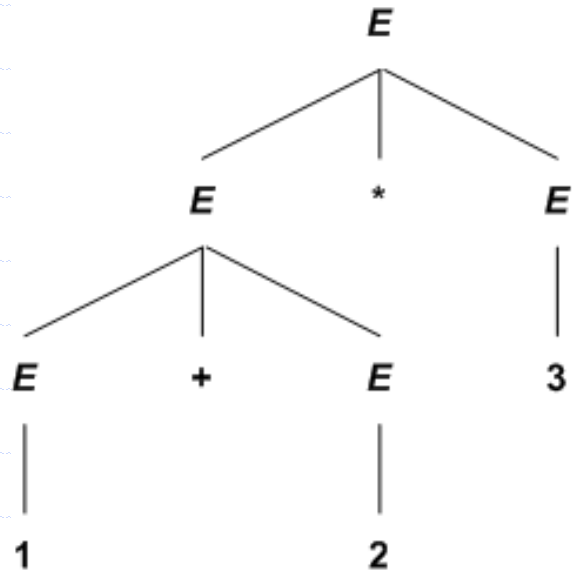


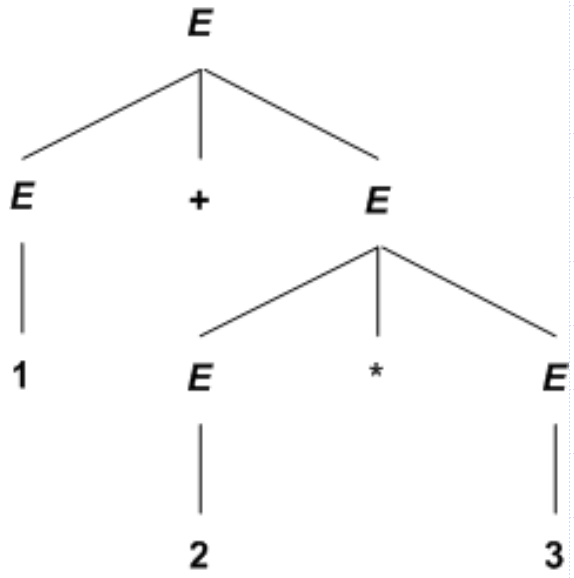


Други пример различитих значења!

- Пример 2: Граматика „Калкулатор 1”
  - Два стабла за реченицу „1+2*3”
- Ако се стабло користи за интерпретирање значења
  - Прво значење: (1+2)*3 = 9
  - Друго значење: 1+(2*3) = 7

$E \rightarrow \mathbf{id}$
$E \rightarrow \mathbf{num}$
$E \rightarrow E * E$
$E \rightarrow E / E$
$E \rightarrow E + E$
$E \rightarrow E - E$
$E \rightarrow (E)$





Решење проблема: Јасна граматика „Калкулатор 2”

- Задатак: Покушајмо да пронађемо јасну граматику „Калкулатор 2”, која дефинише (прихвата) исти језик
- Решење:
  - Прво, оператор „*” треба да има првенство у односу на „+”
  - Друго, сви оператори треба да буду асоцијативни у лево
  - Ово се постиже увођењем нових нетерминалних симбола T и F
  - Да би „*” била асоцијативна у десно, уместо ове продукције: „T → T * F” писали би ову продукцију „T → F * T”.

$E \rightarrow E + T$	$T \rightarrow T * F$	$F \rightarrow \mathbf{id}$
$E \rightarrow E - T$	$T \rightarrow T / F$	$F \rightarrow \mathbf{num}$
$E \rightarrow T$	$T \rightarrow F$	$F \rightarrow (E)$

Bison: генератор синт. анализатора
Генерише Це код на основу улазне спецификације

- Bison је наследник алата Yacc
  - Улазна спец. се састоји од три дела, раздвојених са %{, %} и %% (врло слично Flex-у)
    - Програмски код: Це код који ће користи акције из доњих делова
    - Декларације: списак терминала, нетерминала, итд.
    - Граматичка правила (продукције):
      нетерминал : дефиниција, нпр.
      - exp : exp PLUS exp {семантичка акција}
      - где је exp нетерминал (израз) а PLUS терминал (оператор +)
  - Излаз је Це код: y.tab.h и y.tab.c
  - Приликом генерисања води рачуна о конфликтима

Пример спецификације (без акција)

```
%{
int yylex(void);
void error(char* s) {...}
%}
%token ID WHILE BEGIN END DO IF THEN
       ELSE SEMI ASSIGN
%start prog
%%
prog: stmlist
stm: ID ASSIGN ID                  S → id := id
   | WHILE ID DO stm               S → while id do S
   | BEGIN stmlist END             S → begin L end
   | IF ID THEN stm                S → if id then S
   | IF ID THEN stm ELSE stm       S → if id then S else S
stmlist: stm                       L → S
       | stmlist SEMI stm          L → L ; S
```

Синтаксни анализатор за Tiger (1/6)

tiger.grm (1/3)

```
%union {
        int pos;
        int ival;
        string sval;
        }
%token <sval> ID STRING
%token <ival> INT

%token
  COMMA COLON SEMICOLON LPAREN RPAREN LBRACK RBRACK
  LBRACE RBRACE DOT
  PLUS MINUS TIMES DIVIDE EQ NEQ LT LE GT GE
...
%nonassoc EQ NEQ LT LE GT GE
%left MINUS PLUS
%left TIMES DIVIDE
%left DOT
%left NEG
```

Синтаксни анализатор за Tiger (2/6)

tiger.grm (2/3)

```
%start input

%%
input:  stm
;
stm:    assignment
      | ifthenelse
      | ifthen
      | while
      | for
      | break
      | compoundstm
      | let
;
…
```

```
exp:    lvalue
      | NIL
      | INT
      | STRING
      | NEG INT
      | functioncall
      | arithmetic
      | comparison
      | booleanexp
      | recordcreation
      | arraycreation
;
…
```

Синтаксни анализатор за Tiger (3/6)

tiger.grm (3/3)

```
arithmetic: exp PLUS exp
        | exp MINUS exp
        | exp TIMES exp
        | exp DIVIDE exp
        | MINUS exp %prec NEG
;
comparison: exp EQ exp
        | exp NEQ exp
        | exp GT exp
        | exp LT exp
        | exp GE exp
        | exp LE exp
;
booleanexp: exp AND exp
        | exp OR exp
;
…
```

```
decs: /* empty */
        | decs dec
;
dec:      tydec
        | vardec
        | fundec
;
tydec: TYPE ID EQ ty
;
ty:       ID
        | LBRACE tyfields
RBRACE
        | ARRAY OF ID
;
tyfields: /* empty */
        | typeidlist
;
…
```

Синтаксни анализатор за Tiger (4/6)

Изгенерисани y.tab.h

```
/* Tokens.  */
#ifndef YYTOKENTYPE
#define YYTOKENTYPE

   enum yytokentype {
     ID = 258,
     STRING = 259,
     INT = 260,
     COMMA = 261,
     COLON = 262,
     SEMICOLON = 263,
     LPAREN = 264,
     RPAREN = 265,
     LBRACK = 266,
     RBRACK = 267,
     LBRACE = 268, ...
```

Синтаксни анализатор за Tiger (5/6)

parsetest.c – програм за тестирање

```
#include <stdio.h>
#include <stdlib.h>
#include "util.h"
#include "errormsg.h"
…
int main(int argc, char** argv) {
 yyin = stdin; yyout = stdout;
 parse(argv[1]);
 return 0;
}
…
void parse(char* fname) {
EM_reset(fname);
 if (yyparse() == 0) /* parsing worked */
   fprintf(stderr, "Parsing successful!\n");
 else fprintf(stderr, "Parsing failed\n");
}
```

Синтаксни анализатор за Tiger (6/6)

Добро је почети са једноставним тестовима

```
pera
pera.mika
pera[djoka]
pera[mika.djoka]
nil
()
(pera;mika)
(pera;mika;djoka)
100
"string"
-100
foo()
foo(pera)
foo(pera, mika)
pera + mika
pera - mika
pera * mika
pera / mika
```

```
-pera + mika
pera = mika
pera <> mika
pera > mika
pera < mika
pera >= mika
pera <= mika
"string1" = "string2"
pera & mika
pera | mika
pera & mika | djoka
(pera | mika) & djoka
pera {}
pera {mika=djoka}
pera {mika=djoka, laza=nil}
type tippera = perintip
type tippera = {}
type tippera = {id1:id1tip}
```

Bison у практичној употреби

- Предности:
  - Једноставније специфицирати језик помоћу граматике
  - Ако је граматика ваљана, сигурније је да ће и анализатор бити ваљан
  - Алат нам помаже у откривању неких концептуалних проблема у граматици
- Мане:
  - Резултујући анализатор је (мало) спорији од ручно писаног
  - Теже организовање смисленог пријављивања грешака
  - Лоше ношење са нејасним граматикама
  - Теже дебаговање саме граматике (метајезик за описивање граматике уноси индирекцију)

Bison у практичној употреби

- Тренутно се користи за Ruby, PHP, Bash...
- Користио се за GCC до 2004.
- CLANG (Це/Це++ предњи део за LLVM) је одмах писан ручно
- EDG предњи део се користи у већини комерцијалних Це/Це++ компајлера (Intel, пример)
- Постоје и други алати: ANTLR, JavaCC...